

Temperature Sensor Based LED Control

Using TMP36

1. Abstract

This project details the design and simulation of a temperature-based automation and alert system utilizing an Arduino Uno microcontroller and a TMP36 analog temperature sensor. The primary objective is to establish a threshold-driven control mechanism that processes real-time environmental data to actuate an output device. The TMP36 sensor continuously transduces ambient thermal energy into a proportional analog voltage, which the Arduino's Analog-to-Digital Converter (ADC) processes to calculate the temperature in degrees Celsius. A conditional logic framework is implemented: when the ambient temperature exceeds the predefined threshold of 30°C, the system triggers a visual alert by activating an LED; conversely, at or below 30°C, the LED remains deactivated. The entire hardware logic and programming were successfully implemented and validated using the Tinkercad simulation platform

2. Objective

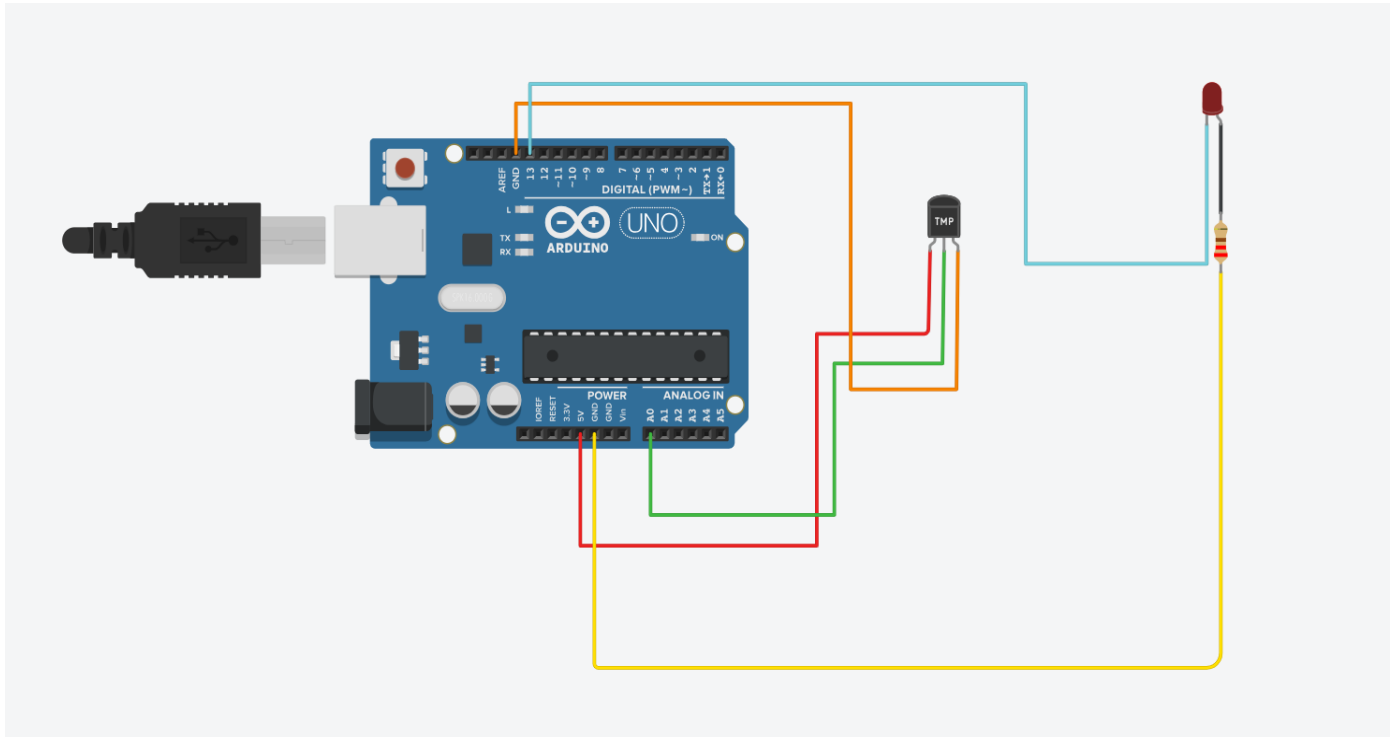
- To interface an analog temperature sensor (TMP36) with a microcontroller for real-time environmental monitoring.
- To process raw analog ADC values into meaningful engineering units (Celsius) using mathematical scaling and sensor specifications.
- To implement threshold-based conditional logic to control an actuator (LED) based on sensor inputs.
- To simulate and validate an IoT-based sensor integration system using Tinkercad.

3. Components Required

The following table presents a structured review of 16 peer-reviewed research papers retrieved from IEEE Xplore and Google Scholar. The table is organized in five columns: S.No, Title / Authors / Source, Abstract, Methodology, and Outcome. Publications span the period 2017–2024, encompassing classical machine learning, deep learning, transformer-based, graph neural network, and multi-modal approaches to fake news detection.

COMPONENT	QUANTITY	DESCRIPTION/ROLE
Arduino Uno	1	Central processing unit; handles ADC conversion, logic execution, and digital output control.
TMP36 Temperature Sensor	1	Analog transducer; converts ambient temperature into a proportional linear voltage output.
LED	1	Visual indicator; serves as an actuator to signal an over-temperature condition.
220Ω Resistor	1	Current-limiting component; protects the LED from excess current draw.
Jumper Wires	As required	Establish electrical pathways between the microcontroller, sensor, and output components.
Tinkercad	1	Online simulation environment for circuit design and Arduino code compilation.

4. Circuit Diagram



5. Circuit Connections

A systematic wiring approach is crucial for accurate sensor readings and actuator control. The connections are outlined below:

COMPONENT PIN	ARDUINO PIN	CONNECTION PURPOSE
TMP36 Left Pin (VCC)	5V	Powers the sensor within its operational voltage range (2.7V to 5.5V).

COMPONENT PIN	ARDUINO PIN	CONNECTION PURPOSE
TMP36 Middle Pin	A0	Transmits the analog voltage output proportional to the temperature to the ADC.
TMP36 Right Pin (GND)	GND	Completes the electrical circuit and provides a common ground reference.
LED Anode (+)	Digital 13	Receives the HIGH/LOW digital signal from the microcontroller to toggle the LED state.
LED Cathode (-)	220Ω → GND	Limits current flow to safe operating levels for the LED (approx. 13.6mA at 5V) and grounds the circuit.

6. Arduino Program

The firmware is developed to continuously poll the sensor, process the data, and execute control logic with a 1-second sampling interval.

```
// Temperature Controlled LED using TMP36
int sensorPin = A0;    // TMP36 middle pin connected to A0
int ledPin = 13;       // LED connected to Digital Pin 13
void setup()
{
  pinMode(ledPin, OUTPUT);
  Serial.begin(9600);
}
void loop()
{
  // Read sensor value
  int sensorValue = analogRead(sensorPin);
  // Convert ADC value to voltage
  float voltage = sensorValue * (5.0 / 1023.0);
  // Convert voltage to temperature (TMP36 formula)
  float temperatureC = (voltage - 0.5) * 100.0;
  // Display temperature in Serial Monitor
  Serial.print("Temperature: ");
  Serial.print(temperatureC);
  Serial.println(" °C");
  // Control LED
  if (temperatureC > 30)
  {
    digitalWrite(ledPin, HIGH); // LED ON
    Serial.println("LED ON");
  }
  else
  {
    digitalWrite(ledPin, LOW); // LED OFF
    Serial.println("LED OFF");
  }
  delay(1000); // Wait 1 second
}
```

7. Working Principle

The operational workflow of the system is based on a continuous sense-process-actuate loop:

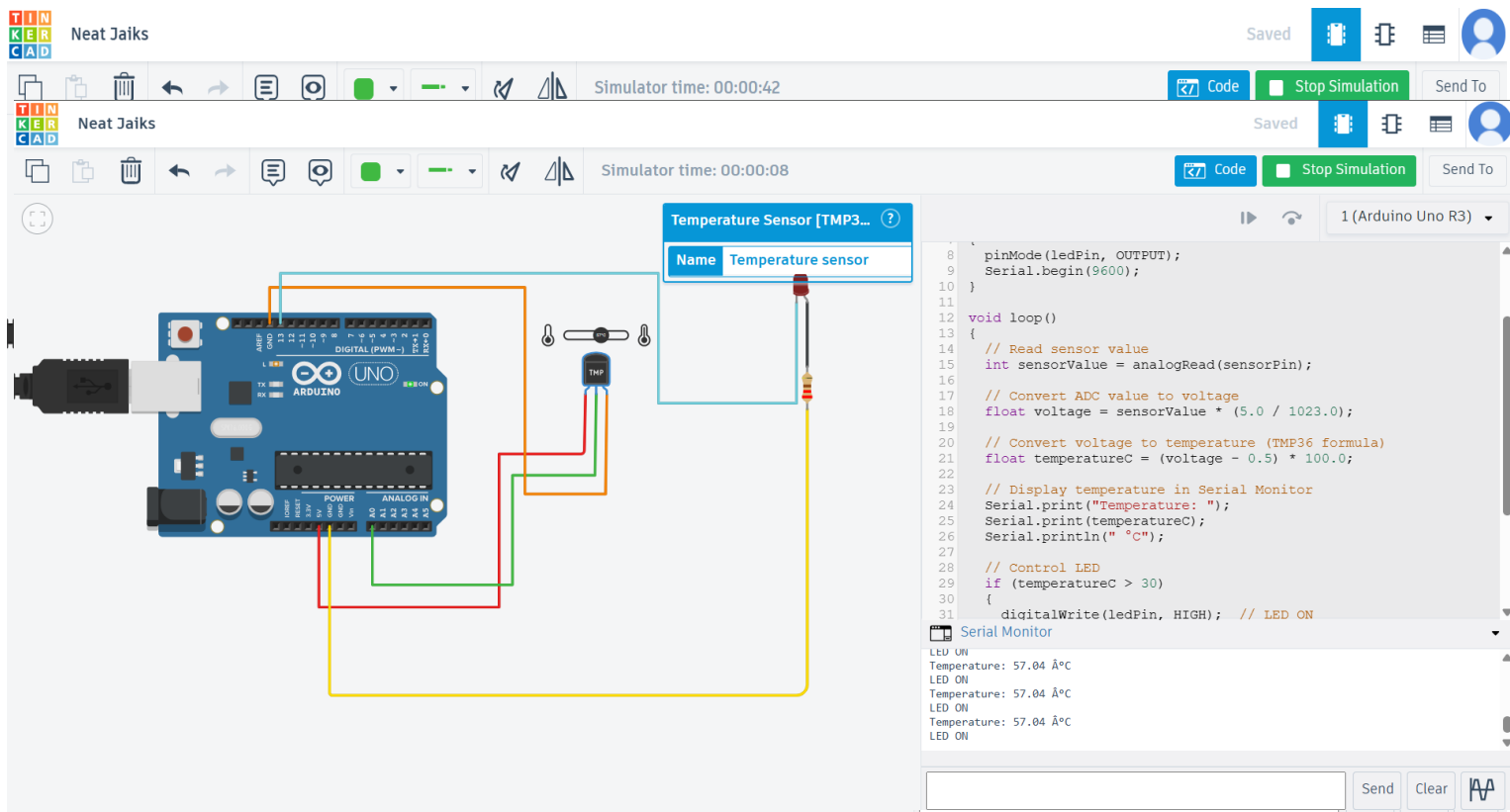
1. **Data Acquisition:** The TMP36 sensor measures the ambient temperature and generates an analog DC voltage at its output pin. The sensor operates on a linear scale factor of 10mV per degree Celsius, with an offset of 500mV at 0°C.

2. Analog-to-Digital Conversion: The Arduino's 10-bit ADC reads the analog voltage on pin A0, converting it into a discrete digital value ranging from 0 to 1023.
3. Mathematical Processing: The microcontroller computes the actual temperature by first scaling the ADC value back to a voltage ($\text{ADC} * 5.0 / 1023.0$) and then applying the TMP36 transfer function $[(\text{Voltage} - 0.5) * 100]$ to yield the temperature in Celsius.
4. Decision & Actuation: The system evaluates the calculated temperature against the 30°C threshold. If the value exceeds 30°C, digital pin 13 is driven HIGH, forward-biasing the LED to indicate an alert state. If the temperature is 30°C or lower, pin 13 is driven LOW, keeping the LED deactivated.

8. Results

The simulation was executed in Tinkercad, and the system's response was observed across varying temperature inputs.

Screenshot 1: Normal Operating Condition (LED OFF)



Screenshot 2: Alert Operating Condition (LED ON)

Observations:

During the simulation, when the TMP36 sensor value was adjusted to simulate an ambient temperature of 28°C, the system calculated the correct value, output "LED OFF" to the serial monitor, and the LED remained

unlit. When the simulated temperature was increased beyond the 30°C threshold (e.g., to 32°C), the Arduino immediately registered the change, drove Pin 13 HIGH, and illuminated the LED. The Serial Monitor accurately reflected the real-time temperature and the corresponding LED state. The system performed reliably with no latency or logic failures.

9. Applications

This fundamental concept of threshold-based sensor actuation is scalable to various real-world IoT and automation scenarios:

- **Smart Home Automation:** Triggering ceiling fans, exhaust systems, or HVAC units when room temperatures deviate from comfort zones.
- **Industrial Safety Monitoring:** Activating alarms or cooling subsystems in server rooms, manufacturing plants, or chemical storage units to prevent overheating.
- **Cold Chain Logistics:** Ensuring perishable goods and pharmaceuticals remain within safe temperature limits during transit.
- **Environmental Monitoring:** Deploying remote alert systems for monitoring forest fire risks or agricultural greenhouse conditions.

10. Conclusion

This project successfully demonstrated the design and implementation of a temperature-based control system using the Arduino Uno and the TMP36 sensor. The simulation, conducted via Tinkercad, validated the system's ability to accurately read, process, and convert analog sensor data into actionable digital outputs. The LED responded precisely to the programmed temperature threshold, confirming the reliability of both the circuit design and the embedded logic. This task provided foundational experience in IoT sensor integration, analog signal processing, and automated control systems, which are critical competencies in the field of embedded electronics and IoT engineering.